PROIECT FINAL

DUMITRESCU FLAVIUS VIRGIL

04.09.2024

PARTEA I - NOTIUNI TEORETICE

1. EXPLICAȚI PE SCURT CE SUNT CERINȚELE DE BUSINESS, LA CE NE FOLOSESC ȘI CINE LE CREEAZĂ

Sunt specificații detaliate create de echipele de management sau de proprietarii afacerilor; descrieri detaliate ale funcționalităților, performanței și comportamentului așteptat al software-ului. Cerințe pot include funcționalități specifice, interacțiuni cu utilizatorii, performanță, securitate și compatibilitate

2. DIFERENȚA ÎNTRE TEST CONDITION SI TEST CASE

Test condition: Acesta reprezintă un aspect al unei componente sau al unui sistem care poate fi verificat prin intermediul unui test case. De exemplu, poate fi vorba despre o funcție, o funcționalitate sau un atribut specific. În esență, este ceea ce veți testa.

Test case: Acesta reprezintă o serie de pași pe care îi veți executa pentru a verifica o anumită funcționalitate. Prin urmare, este modalitatea prin care veți testa respectivul aspect.

3. ENUMERAȚI ȘI EXPLICAȚI PE SCURT ETAPELE PROCESULUI DE TESTARE

- Planificarea testării: Definirea strategiei și scopului testării.
- Analiza și proiectarea testelor: Identificarea cerințelor și proiectarea testelor corespunzătoare.
- Implementarea testelor: Crearea și configurarea mediului de testare și a cazurilor de test.
- Executarea testelor: Rularea cazurilor de test și înregistrarea rezultatelor.
- Evaluarea criteriilor de ieșire: Determinarea dacă testarea este completă.
- Raportarea: Documentarea și comunicarea rezultatelor testării.
- Închiderea testării: Finalizarea activităților de testare și arhivarea rezultatelor.

Mai multe detalii se regasesc in slide-urile de mai jos.

4. EXPLICAȚI DIFERENȚA ÎNTRE RETESTING ȘI REGRESSION TESTING

- Retesting este procesul de testare a unei componente sau funcționalități după ce a fost remediată o eroare. Regression testing este testarea sistemului pentru a se asigura că modificările recente nu au afectat funcționalitățile existente.

PARTEA I - NOTIUNI TEORETICE

5. EXPLICAȚI DIFERENȚA ÎNTRE FUNCTIONAL TESTING ȘI NON-FUNCTIONAL TESTING

- Functional testing verifică dacă sistemul îndeplinește cerințele specificate, concentrându-se pe ce face sistemul. Non-functional testing evaluează calitățile sistemului, cum ar fi performanța, securitatea și utilizabilitatea, concentrându-se pe cum funcționează sistemul.

6. EXPLICAȚI DIFERENȚA ÎNTRE BLACKBOX TESTING ȘI WHITEBOX TESTING

- Blackbox testing testează funcționalitatea sistemului fără a cunoaște structura internă a codului. Whitebox testing implică testarea structurii interne și a fluxurilor logice ale sistemului, necesitând cunoștințe minime de programare.

7. ENUMERAȚI TEHNICILE DE TESTARE ȘI GRUPAȚI-LE ÎN FUNCȚIE DE CATEGORIE (BLACKBOX, WHITEBOX, EXPERIENCE-BASED)

- Tehnici de testare:
  - Blackbox: Equivalence partitioning, Boundary value analysis, Decision table testing, State trasnistion testing.
  - Whitebox: Statement coverage, Branch coverage, Path coverage.
  - Experience-based: Exploratory testing, Error guessing.

8. EXPLICAȚI DIFERENȚA ÎNTRE VERIFICATION ȘI VALIDATION

Verification: Acesta reprezintă procesul de evaluare prin care se determină dacă un produs sau sistem respectă specificațiile și cerințele stabilite. Cu alte cuvinte, verificarea se concentrează pe a verifica dacă ceea ce a fost construit corespunde cu ceea ce a fost planificat și specificat.

Validation: Acesta este procesul de evaluare prin care se asigură că un produs sau sistem îndeplinește nevoile și așteptările utilizatorilor finali. În esență, validarea se concentrează pe a verifica dacă ceea ce a fost construit este util și funcțional pentru utilizatori.

PARTEA I - NOTIUNI TEORETICE

- 9. •EXPLICAȚI DIFERENȚA ÎNTRE POSITIVE TESTING ȘI NEGATIVE TESTING ȘI DAȚI CÂTE UN EXEMPLU DIN FIECARE
- Positive testing verifică dacă sistemul funcționează corect cu date valabile (ex: introducerea unei parole corecte).
- Negative testing verifică dacă sistemul gestionează adecvat date invalide (ex: introducerea unei parole greșite și verificarea mesajului de eroare)
- 10. ENUMERAȚI ȘI EXPLICAȚI PE SCURT NIVELURILE DE TESTARE
- • Nivelurile de testare includ:
- o Unit testing: Testarea componentelor individuale ale codului.
- o Integration testing: Testarea interacțiunilor dintre componente.
- o System testing: Testarea întregului sistem integrat pentru a verifica conformitatea cu cerințele specificate.
- o Acceptance testing: Testarea realizată de utilizatori finali pentru a verifica dacă sistemul îndeplinește nevoile și așteptările acestora.

PARTEA I - NOTIUNI TEORETICE

Etapele procesului de testare

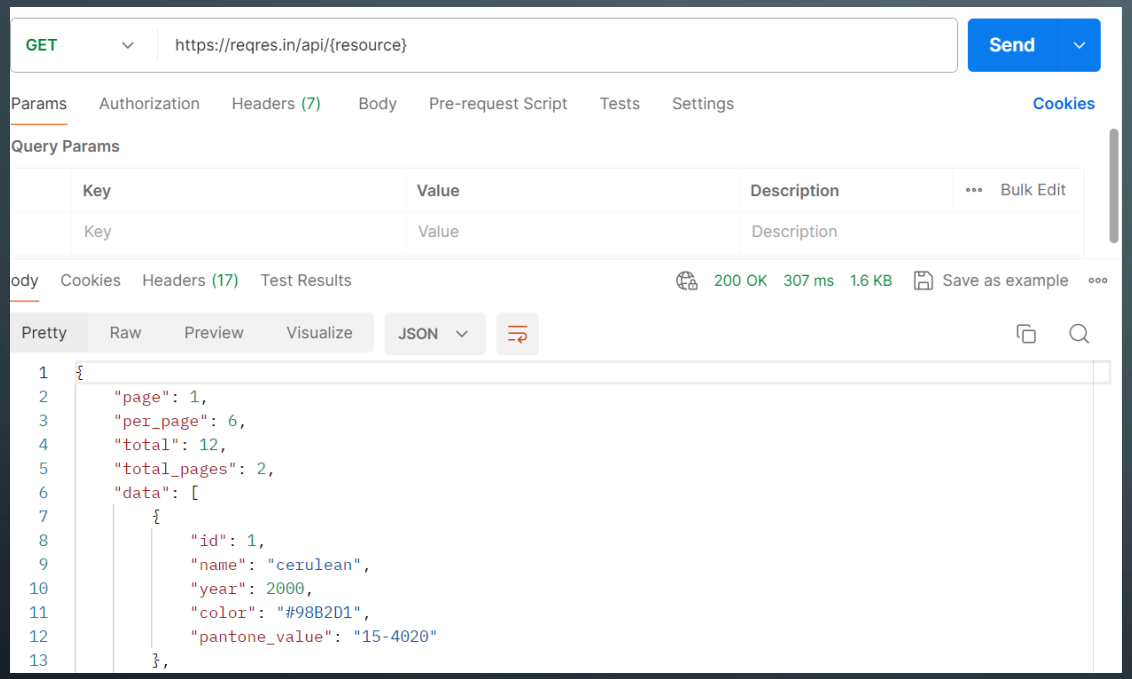
Monitorizarea progresului testării, controlul activităților de testare, gestionarea resurselor și rezolvarea problemelor apărute.

Reprezintă testarea celei mai mici bucăți funcționale dintr-o aplicație cum ar fi: funcții, clase, proceduri, interefețe

PARTEA II - ASPECTE PRACTICE

Testarea API in Postman – site si documentatie https://reqres.in/api.

Metoda GET



Status 200 OK – The request succeeded
307 ms – timpul de raspuns

GET (Obține):
Rol: Solicită informații de la un server.
Descriere: Comanda GET este utilizată pentru a obține date de la un URL specific. Răspunsul conține informațiile cerute, cum ar fi pagini web, imagini sau alte resurse

PARTEA II - ASPECTE PRACTICE

Metoda GET – lista users

GET https://reqres.in/api/users Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	••• Bulk Edit
Key	Value	Description	

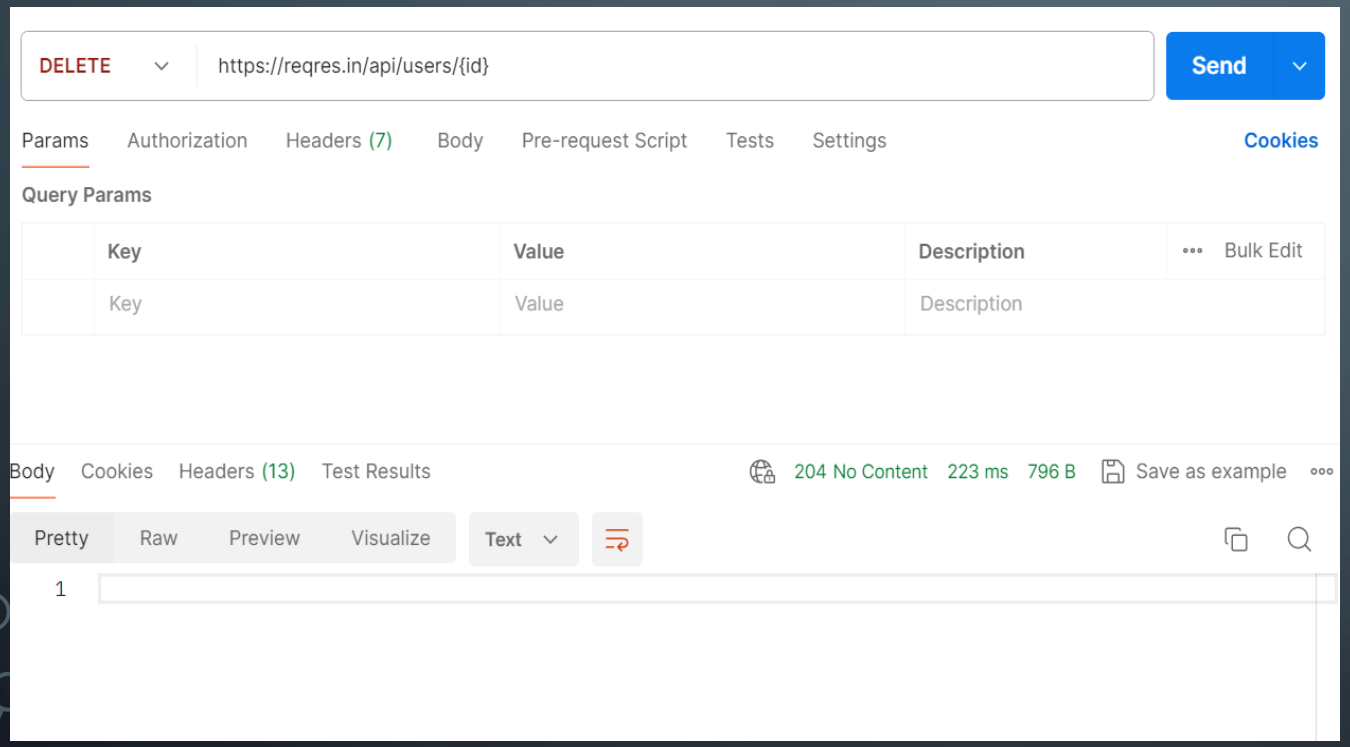
Body Cookies Headers (17) Test Results 200 OK 287 ms 1.88 KB Save as example

Pretty Raw Preview Visualize JSON


```
1  {
2      "page": 1,
3      "per_page": 6,
4      "total": 12,
5      "total_pages": 2,
6      "data": [
7          {
8              "id": 1,
9              "email": "george.bluth@reqres.in",
10             "first_name": "George",
11             "last_name": "Bluth",
12             "avatar": "https://reqres.in/img/faces/1-image.jpg"
```

PARTEA II - ASPECTE PRACTICE

Metoda DELETE – Delete user



DELETE (Șterge):

Rol: Șterge o resursă de pe server.

Descriere: Comanda DELETE este folosită pentru a șterge o resursă specifică de pe server. De exemplu, poți șterge un articol dintr-un blog sau un utilizator dintr-o bază de date.

PARTEA II - ASPECTE PRACTICE

Metoda GET – resource – identifica resursa/situl

GET https://reqres.in/api/{resource} Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	••• Bulk Edit
Key	Value	Description	

Body Cookies Headers (18) Test Results 200 OK 25 ms 1.61 KB Save as example •••

Pretty Raw Preview Visualize JSON


```
1  {
2      "page": 1,
3      "per_page": 6,
4      "total": 12,
5      "total_pages": 2,
6      "data": [
7          {
8              "id": 1,
9              "name": "cerulean",
10             "year": 2000,
```

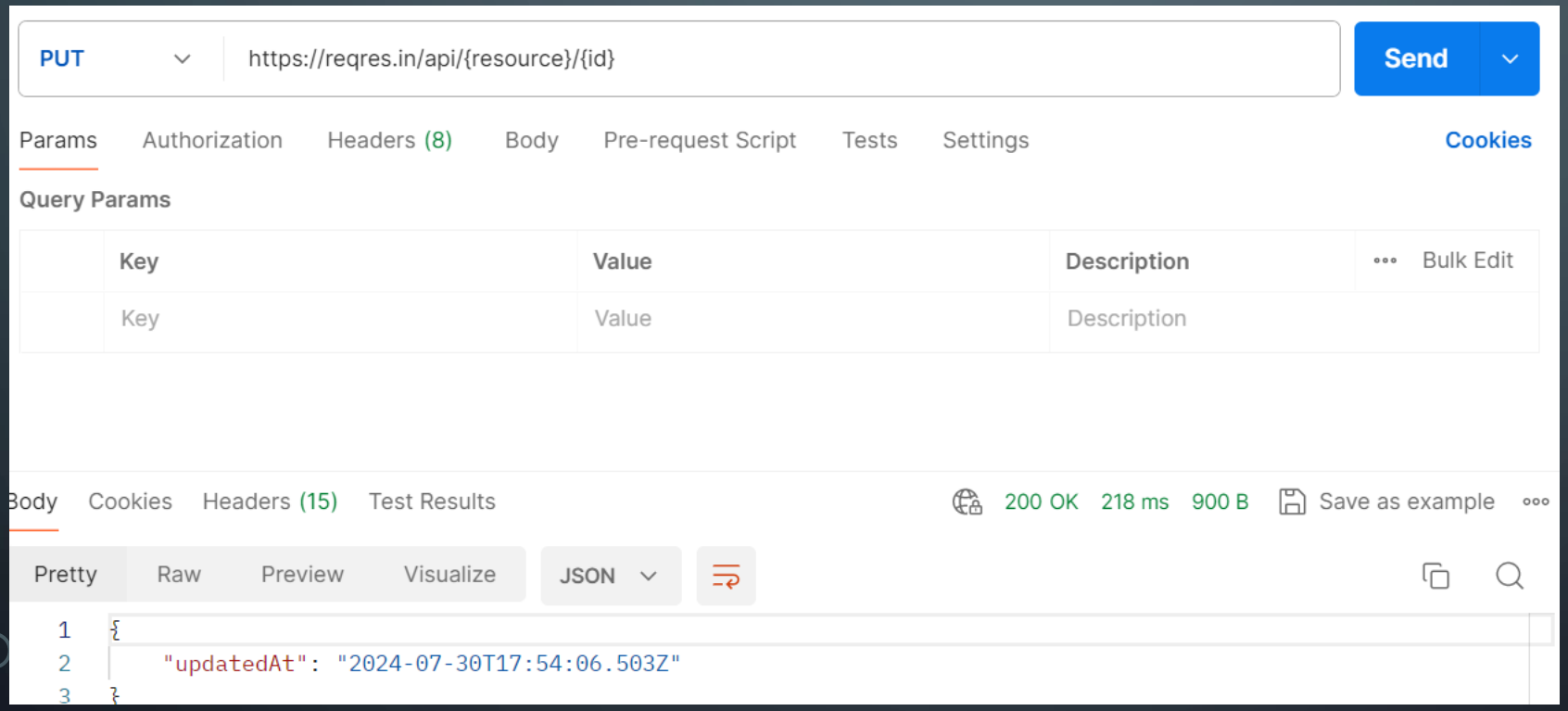


```
{
    "id": 1,
    "name": "cerulean",
    "year": 2000,
    "color": "#98B2D1",
    "pantone_value": "15-4020"
},
```


Rezultatul adus de comnda in urma interogarii. Aici avem detaliile unei reurse/site in limbaj de programare JSON.

PARTEA II - ASPECTE PRACTICE

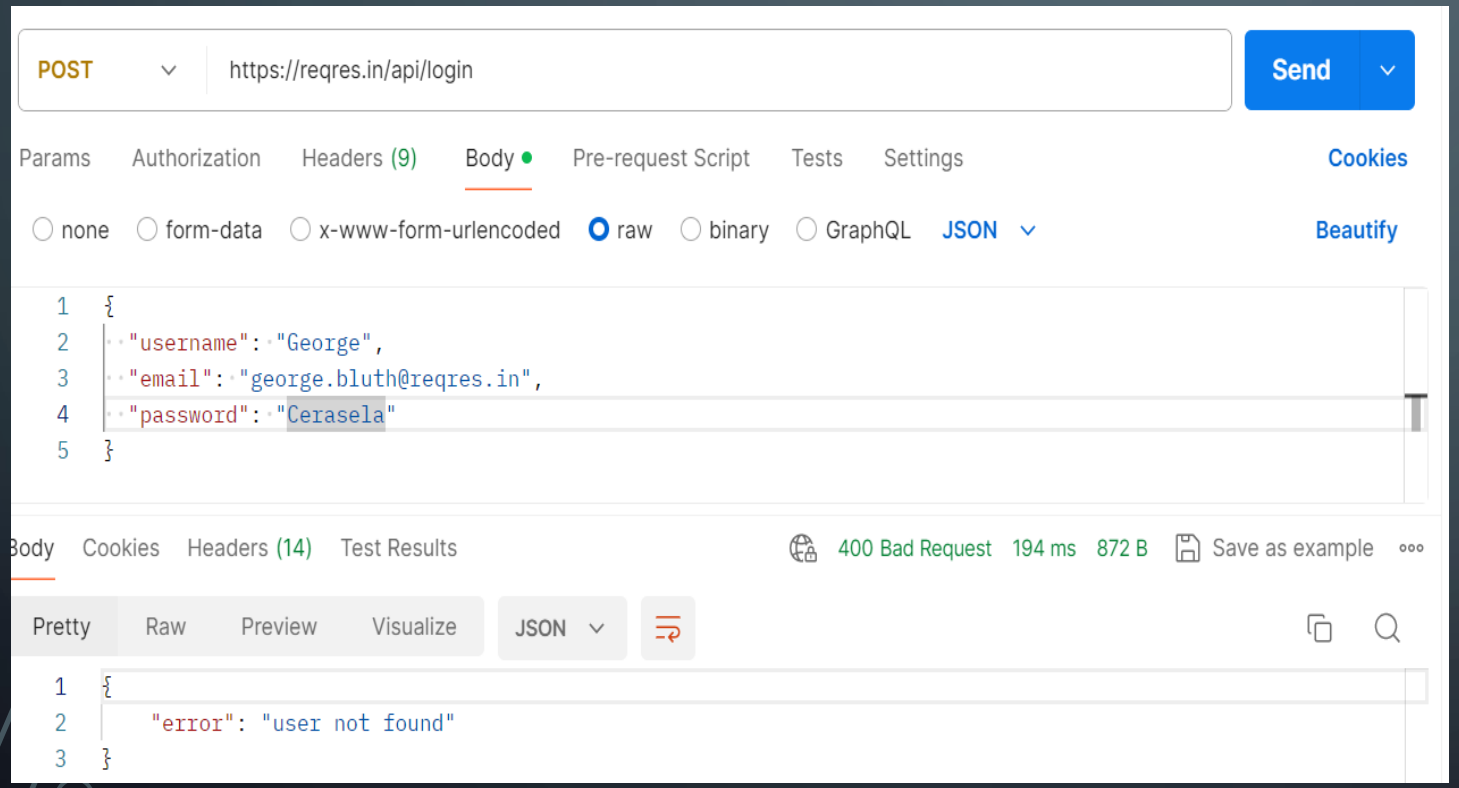
Comanda PUT – user ID



PUT (Înlocuiește):
Rol: Înlocuiește complet o resursă existentă sau o creează dacă nu există.
Descriere: Comanda PUT este folosită pentru a înlocui complet o resursă existentă cu una nouă. Dacă resursa nu există, este creată. Este utilizată pentru actualizări complete.

PARTEA II - ASPECTE PRACTICE

Comanda POST – user not found



1. **POST (Trimite):**
   - **Rol:** Trimite date către un server pentru a fi procesate.
   - **Descriere:** Comanda POST este folosită pentru a trimite date către server, de exemplu, pentru a crea un nou obiect sau a efectua o acțiune. De obicei, este utilizată în formulare web sau pentru a adăuga date într-o bază de date.

PARTEA II - ASPECTE PRACTICE

Metoda POST – Creare user

POST https://reqres.in/api/users/{id} Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "username": "string",
3   "email": "string",
4   "password": "string"
5 }
```

Body Cookies Headers (14) Test Results 201 Created 237 ms 951 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2     "username": "string",
3     "email": "string",
4     "password": "string",
5     "id": "611",
6     "createdAt": "2024-07-28T16:02:01.479Z"
7 }
```

PARTEA II - ASPECTE PRACTICE

Metoda POST – Creare user

POST https://reqres.in/api/users/{id} Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "username": "string",
3   "email": "string",
4   "password": "string"
5 }
```

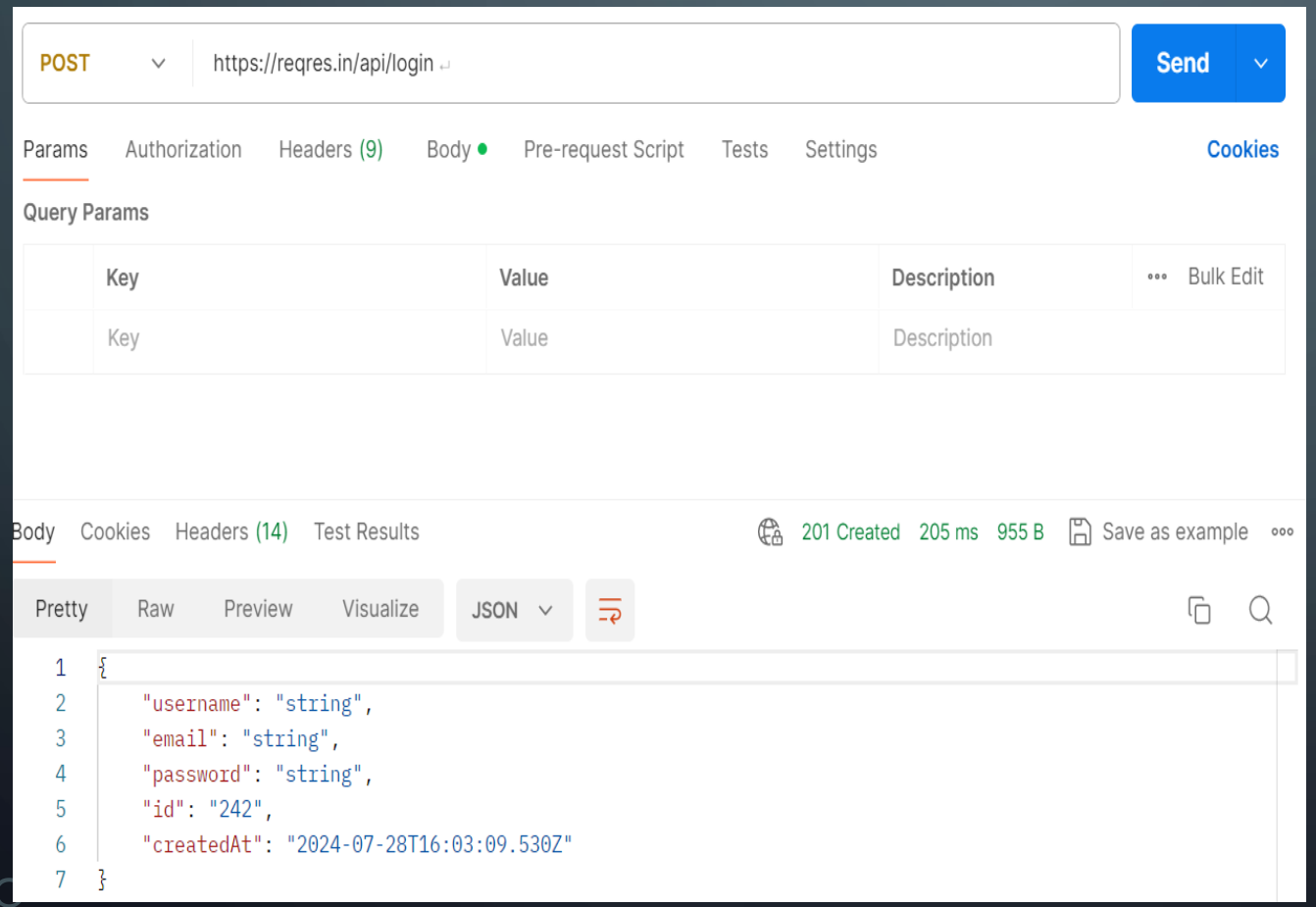
Body Cookies Headers (14) Test Results 201 Created 237 ms 951 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2     "username": "string",
3     "email": "string",
4     "password": "string",
5     "id": "611",
6     "createdAt": "2024-07-28T16:02:01.479Z"
7 }
```

PARTEA II - ASPECTE PRACTICE

Metoda POST – Login



CONCLUZII

1. Validarea API-ului:

•**Funcționalitate:** API-ul a fost testat în detaliu pentru a se asigura că toate funcționalitățile sale corespund specificațiilor inițiale.

•**Erori și rezolvarea lor:** Au fost identificate și corectate diverse bug-uri și erori care ar fi putut afecta funcționarea aplicației.

2. Performanța:

•**Timp de răspuns:** Testele de performanță au arătat un timp de răspuns adecvat pentru majoritatea cererilor, asigurând astfel o experiență de utilizator fluentă.

•**Stabilitate:** API-ul a demonstrat stabilitate și fiabilitate sub diferite condiții de încărcare.

LESSONS LEARNED

- **Procesul de testare:** Importanța unei metodologii riguroase de testare pentru asigurarea calității produsului final.

MULTUMESC PENTRU ATENTIE!